



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

BIOMETRYCZNE SYSTEMY ZABEZPIECZEŃ

System multimodalnej identyfikacji użytkownika
MultiVerify

Kacper Ręczak

Kamil Szopniewski

Nr albumu: 134968, 134976

Kierunek studiów: Informatyka

Prowadząca: mgr inż. Ewa Żesławska

Rzeszów, 2026

Dziękujemy mgr inż. Ewie Żesławskiej za prowadzenie przedmiotu Biometryczne systemy zabezpieczeń oraz za udostępnienie szablonu dokumentacji.

Spis treści

1. Wprowadzenie	7
1.1. Cel projektu	7
1.2. Kontekst zastosowania	7
1.3. Zakres realizacji	7
1.4. Struktura dokumentu	7
2. Opis problemu i założeń	8
2.1. Opis problemu	8
2.2. Dane wejściowe	8
2.3. Założenia działania	8
2.4. Ograniczenia	9
3. Analiza istniejących rozwiązań	10
3.1. Wprowadzenie	10
3.2. Face ID (Apple)	10
3.3. Rozpoznawanie mówcy (speaker recognition)	10
3.4. Systemy komercyjne multimodalne	10
3.5. Nowość projektu MultiVerify	10
4. Opis koncepcji rozwiązania	12
4.1. Ogólna idea	12
4.2. Schemat blokowy	12
4.3. Etap rejestracji	12
4.4. Etap identyfikacji	13
4.5. Parametry decyzyjne	13
5. Zastosowane metody i algorytmy	14
5.1. Podobieństwo cosinusowe	14
5.2. Modalność twarzy (DeepFace / Facenet)	14
5.2.1. Ekstrakcja cech	14
5.2.2. Uzasadnienie wyboru	14
5.3. Modalność głosu (MFCC)	14
5.3.1. Ekstrakcja cech	14
5.3.2. Uzasadnienie wyboru	14
5.4. Fuzja wyników (late fusion)	15
5.4.1. Suma ważona	15
5.4.2. Reguła AND	15
5.4.3. Uzasadnienie fuzji	15
6. Implementacja	16
6.1. Środowisko	16

6.2.	Struktura projektu	16
6.3.	Interfejs modalności	16
6.4.	Silnik fuzji	16
6.5.	Identyfikacja użytkownika.....	17
6.6.	Rejestracja	17
6.7.	Testy.....	17
7.	Wyniki i eksperymenty	18
7.1.	Konfiguracja eksperymentu.....	18
7.2.	Przykład identyfikacji.....	18
7.3.	Analiza wyników	18
7.3.1.	Modalność twarzy	18
7.3.2.	Modalność głosu	18
7.3.3.	Fuzja multimodalna	19
7.3.4.	Wpływ progu	19
7.4.	Analiza bezpieczeństwa.....	19
7.4.1.	Zagrożenia.....	19
7.4.2.	Środki zaradcze	19
7.5.	Podsumowanie eksperymentów	19
8.	Wnioski końcowe	20
8.1.	Podsumowanie.....	20
8.2.	Najważniejsze obserwacje.....	20
8.3.	Ocena skuteczności	20
8.4.	Możliwości rozwoju	20
8.5.	Podział pracy	20
	Bibliografia	21
	Spis rysunków	22
	Spis listingów	23

1. Wprowadzenie

1.1. Cel projektu

Celem projektu jest zaprojektowanie i implementacja systemu *MultiVerify* — rozwiązania biometrycznego wykorzystującego **dwie modalności**: rozpoznawanie twarzy oraz rozpoznawanie mówcy. System realizuje identyfikację użytkownika w trybie $1:N$ (porównanie z bazą zarejestrowanych osób) oraz podejmuje decyzję o przyznaniu lub odmowie dostępu.

Projekt został opracowany w ramach przedmiotu *Biometryczne systemy zabezpieczeń* i wpisuje się w temat *Systemy multimodalne (zaawansowane)*.

1.2. Kontekst zastosowania

Systemy biometryczne są powszechnie stosowane w kontroli dostępu do pomieszczeń, stacji roboczych oraz aplikacji wymagających uwierzytelnienia użytkownika. Pojedyncza cecha biometryczna (np. sam obraz twarzy) może być niewystarczająca w warunkach słabego oświetlenia, szumu akustycznego lub prób ataku fałszywą tożsamością.

Podejście multimodalne łączy niezależne źródła informacji i pozwala podnieść niezawodność decyzji identyfikacyjnej [4, 6]. **MultiVerify** demonstruje tę ideę w praktyce na przykładzie fuzji wyników z modułu twarzy i modułu głosu.

1.3. Zakres realizacji

Zakres projektu obejmuje:

- implementację modułu ekstrakcji cech twarzy (biblioteka DeepFace, model Facenet),
- implementację modułu ekstrakcji cech głosu (współczynniki MFCC),
- silnik fuzji wyników (late fusion) z dwiema strategiami decyzyjnymi,
- proces rejestracji użytkowników oraz identyfikacji $1:N$,
- przeprowadzenie eksperymentów porównawczych (modalność pojedyncza vs. fuzja),
- analizę skuteczności, ograniczeń i zagrożeń bezpieczeństwa.

Poza zakresem pozostaje m.in. integracja z fizycznymi urządzeniami (kamera, bramka), rejestracja na żywo oraz wdrożenie produkcyjne w środowisku rozproszonym.

1.4. Struktura dokumentu

Rozdział 2 opisuje problem i założenia. Rozdział 3 przedstawia analizę istniejących rozwiązań. W rozdziale 4 przedstawiono koncepcję systemu, a w rozdziale 5 — zastosowane metody. Rozdział 6 zawiera opis implementacji. Wyniki eksperymentów i analiza bezpieczeństwa znajdują się w rozdziale 7. Dokument kończy rozdział 8 z wnioskami.

2. Opis problemu i założeń

2.1. Opis problemu

Problem polega na automatycznej identyfikacji użytkownika na podstawie danych biometrycznych w postaci **zdjęcia twarzy** oraz **nagrania głosu**. System musi:

1. zarejestrować użytkowników (utworzyć wzorce biometryczne),
2. porównać nowe dane (probe) z bazą wszystkich użytkowników,
3. wybrać najbardziej prawdopodobną tożsamość,
4. podjąć decyzję o dostępie na podstawie progu akceptacji.

W praktyce występują dwa rodzaje błędów [4]:

FAR fałszywa akceptacja — system uznaje obcą osobę za zarejestrowanego użytkownika,

(False

Ac-

cept

Rate)

—

FRR fałszywe odrzucenie — system odmawia dostępu prawdziwemu użytkownikowi.

(False

Re-

ject

Rate)

—

Celem projektu jest minimalizacja obu metryk przy zachowaniu prostoty implementacji demonstracyjnej.

2.2. Dane wejściowe

System operuje na plikach przechowywanych lokalnie:

- **Twarz:** obrazy `face_*.jpg` lub `face_*.png`,
- **Głos:** nagrania `voice_*.wav` lub `voice_*.mp3`.

Na użytkownika zaleca się co najmniej trzy zdjęcia i trzy nagrania (5–10 s mowy). Pliki probe (np. `face_3`, `voice_3`) powinny być różne od wzorców użytych wyłącznie do testów, aby uniknąć zawyżonych wyników.

2.3. Założenia działania

Przyjęto następujące założenia:

- jedna osoba na zdjęciu; twarz widoczna frontalnie lub lekko z boku,

- nagrania głosu zawierają wyłącznie mowę danej osoby, bez silnego szumu tła,
- identyfikacja odbywa się offline (brak wymogu chmury),
- baza użytkowników jest skończona i znana z góry (tryb 1:N),
- środowisko uruchomieniowe: Python 3.12, system Windows/Linux.

2.4. Ograniczenia

Główne ograniczenia rozwiązania:

- brak rejestracji i identyfikacji w czasie rzeczywistym z kamery/mikrofonu,
- proste cechy głosu (MFCC) — mniejsza rozdzielczość niż w modelach głębokich,
- zależność modułu twarzy od biblioteki DeepFace i modelu Facenet,
- wrażliwość na jakość danych (oświetlenie, format plików audio),
- brak mechanizmów anty-spoofing (np. wykrywania zdjęcia zamiast żywej twarzy).

3. Analiza istniejących rozwiązań

3.1. Wprowadzenie

Na rynku oraz w literaturze naukowej dostępnych jest wiele systemów biometrycznych opartych na pojedynczej lub wielu modalnościach. Poniżej przedstawiono trzy reprezentatywne podejścia oraz wskazano elementy nowatorskie projektu MultiVerify.

3.2. Face ID (Apple)

System Face ID w urządzeniach Apple wykorzystuje zaawansowany czujnik TrueDepth do mapowania geometrii twarzy w trzech wymiarach [1]. Uwierzytelnianie odbywa się lokalnie na urządzeniu, a dane biometryczne nie są przesyłane do chmury w postaci surowej.

Zalety: wysoka wygoda użytkownika, integracja ze sprzętem, mechanizmy anty-spoofing.

Wady: zamknięta implementacja, zależność od ekosystemu Apple, brak jawnej multimodalności z głosem w standardowym API.

3.3. Rozpoznawanie mówcy (speaker recognition)

Klasyczne systemy rozpoznawania mówcy opierają się na ekstrakcji cech akustycznych — w tym MFCC — oraz klasyfikacji lub porównywaniu wektorów cech [2]. Współczesne rozwiązania komercyjne stosują modele głębokie (np. x-vectors), ale podejście MFCC pozostaje popularne w zastosowaniach edukacyjnych i prototypowych [5].

Zalety: niski koszt obliczeniowy, interpretowalność cech.

Wady: wrażliwość na szum, mikrofon i długość wypowiedzi; podatność na atak replay.

3.4. Systemy komercyjne multimodalne

Duże systemy kontroli dostępu (np. w obiektach korporacyjnych) często łączą kartę RFID, PIN oraz biometrię twarzy lub odcisku palca [3]. Fuzja informacji może odbywać się na poziomie decyzji (late fusion) lub cech (early fusion) [6].

Zalety: wysoka niezawodność przy właściwej konfiguracji.

Wady: złożoność wdrożenia, koszt, brak transparentności algorytmów.

3.5. Nowość projektu MultiVerify

Projektowany system wyróżnia się następującymi cechami:

- **otwarta implementacja** w Pythonie z czytelną strukturą modułową,
- **łatwa replikacja** eksperymentów (skrypty rejestracji, identyfikacji i testów),
- **porównanie strategii fuzji** (suma ważona vs. reguła AND) oraz wariantów wag,
- **świadome połączenie** modelu głębokiego (twarz) z metodą klasyczną (głos),
- analiza metryk FAR/FRR na rzeczywistych danych zespołu projektowego.

MultiVerify nie konkuruje z systemami komercyjnymi pod względem bezpieczeństwa produkcyjnego, lecz demonstruje pełny pipeline multimodalny zgodny z wymaganiami akademickimi.

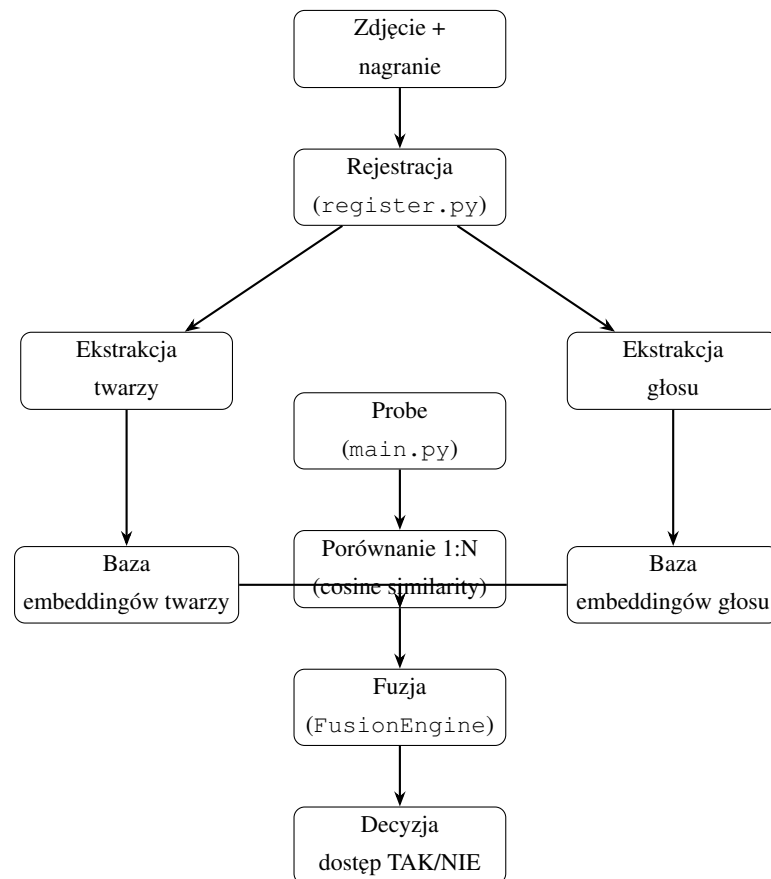
4. Opis koncepcji rozwiązania

4.1. Ogólna idea

System MultiVerify realizuje klasyczny pipeline biometryczny podzielony na dwa główne etapy: **rejestrację (enrollment)** oraz **identyfikację (1:N)**. W obu przypadkach przetwarzane są dwie modalności równolegle; decyzja końcowa zapada po fazie fuzji wyników cząstkowych.

4.2. Schemat blokowy

Na rys. 4.1 przedstawiono przepływ danych w systemie.



Rysunek 4.1. Schemat blokowy systemu MultiVerify.

4.3. Etap rejestracji

Dla każdego użytkownika system:

1. wczytuje pliki `face_*` i `voice_*` z katalogu `data/users/<user>/`,
2. wyznacza embeddingi twarzy (wektor Facenet) oraz wektory MFCC głosu,
3. zapisuje wyniki w plikach `face_embeddings.npy` oraz `voice_embeddings.npy`.

4.4. Etap identyfikacji

Podczas identyfikacji system:

1. ekstrahuje cechy z nowego zdjęcia i nagrania (probe),
2. dla każdego użytkownika w bazie oblicza wynik podobieństwa twarzy i głosu,
3. wybiera najlepszy wynik w ramach wielu wzorców jednego użytkownika,
4. łączy wyniki w silniku fuzji,
5. wskazuje użytkownika z najwyższym wynikiem końcowym i weryfikuje próg akceptacji.

4.5. Parametry decyzyjne

Domyślnie zastosowano:

- wagi fuzji: $w_f = 0,6$, $w_v = 0,4$,
- próg akceptacji: $\tau = 0,7$,
- strategia domyślna: suma ważona (weighted).

Parametry te mogą być modyfikowane z linii poleceń programu `main.py`.

5. Zastosowane metody i algorytmy

5.1. Podobieństwo cosinusowe

Dla dwóch wektorów cech $\mathbf{a}$ i $\mathbf{b}$ stosowany jest współczynnik podobieństwa cosinusowego:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}, \quad (5.1)$$

z obcięciem wyniku do przedziału $[0, 1]$. Metoda ta jest stosowana zarówno w module twarzy, jak i głosu.

5.2. Modalność twarzy (DeepFace / Facenet)

5.2.1. Ekstrakcja cech

Moduł twarzy korzysta z biblioteki DeepFace [8], która udostępnia gotowe modele głębokiego uczenia. W projekcie zastosowano model **Facenet** [7], zwracający embedding o wymiarze 128.

Funkcja `DeepFace.represent` przetwarza obraz wejściowy i zwraca wektor liczb rzeczywistych reprezentujący tożsamość twarzy w przestrzeni metrycznej.

5.2.2. Uzasadnienie wyboru

Facenet zapewnia dobrą jakość rozpoznawania przy relatywnie niskim wysiłku implementacyjnym (brak trenowania własnej sieci). DeepFace upraszcza integrację z projektem demonstracyjnym.

5.3. Modalność głosu (MFCC)

5.3.1. Ekstrakcja cech

Współczynniki MFCC (*Mel-Frequency Cepstral Coefficients*) są standardowym opisem krótkoczasowej energii widma mowy [2]. Dla sygnału audio obliczana jest macierz MFCC o wymiarze $13 \times T$ (liczba ramek czasowych T), a następnie dla każdego współczynnika wyznaczane są:

- średnia wzdłuż osi czasu,
- odchylenie standardowe wzdłuż osi czasu.

Wektor cech głosu ma wymiar 26 (13 średnich + 13 odchyleń). Implementacja wykorzystuje bibliotekę `librosa` [5].

5.3.2. Uzasadnienie wyboru

MFCC charakteryzuje się niskim kosztem obliczeniowym i przejrzystością, co ułatwia analizę w projekcie akademickim. Jednocześnie pozwala zestawić podejście klasyczne (głos) z głębokim (twarz) w systemie multimodalnym.

5.4. Fuzja wyników (late fusion)

5.4.1. Suma ważona

W strategii *weighted* wynik końcowy obliczany jest jako:

$$S = w_f \cdot s_f + w_v \cdot s_v, \quad (5.2)$$

gdzie s_f i s_v to wyniki podobieństwa twarzy i głosu, a $w_f + w_v = 1$. Dostęp przyznawany jest, gdy $S \geq \tau$.

5.4.2. Reguła AND

W strategii *and* decyzja pozytywna wymaga spełnienia obu warunków:

$$s_f \geq \tau \quad \wedge \quad s_v \geq \tau. \quad (5.3)$$

Wynik rankingowy ustawiano na $S = \min(s_f, s_v)$, co ogranicza fałszywe akceptacje przy dominacji jednej, zawyżonej modalności.

5.4.3. Uzasadnienie fuzji

Fuzja na poziomie decyzji (late fusion) jest prostsza w implementacji niż łączenie wektorów cech (early fusion) i umożliwia bezpośrednie porównanie wkładu każdej modalności [6].

6. Implementacja

6.1. Środowisko

Projekt zaimplementowano w języku Python 3.12. Główne biblioteki:

- **numpy** — operacje na wektorach i macierzach,
- **deepface, tf-keras** — moduł twarzy,
- **librosa, soundfile** — przetwarzanie audio,
- **pytest** — testy jednostkowe.

Zalecane jest użycie środowiska wirtualnego (venv). Szczegółowa instrukcja uruchomienia znajduje się w pliku `README.md`.

6.2. Struktura projektu

```
MultiVerify/  
  main.py                # identyfikacja 1:N  
  register.py            # rejestracja użytkowników  
  src/  
    modalities/  
      base.py            # interfejs BiometricModality  
      face/              # FaceModality (DeepFace)  
      voice/             # VoiceModality (MFCC)  
    fusion/  
      fusion_engine.py  
  scripts/run_experiments.py  
  data/users/<user>/      # dane biometryczne (lokalnie)  
  tests/
```

6.3. Interfejs modalności

Każda modalność implementuje klasę bazową z metodami `extract_features` oraz `verify`, co umożliwia jednolite traktowanie różnych źródeł biometrii.

6.4. Silnik fuzji

Listing 6.1 przedstawia fragment implementacji strategii fuzji.

Listing 6.1. Fragment silnika fuzji (FusionEngine).

```
1 def fuse(self, face_score, voice_score):  
2     if self.strategy == "and":  
3         final_score = min(face_score, voice_score)  
4         access_granted = (  
5             face_score >= self.threshold
```

```

6         and voice_score >= self.threshold
7     )
8     else:
9         final_score = (
10             face_score * self.face_weight
11             + voice_score * self.voice_weight
12         )
13         access_granted = final_score >= self.threshold
14
15     return {
16         "final_score": round(final_score, 4),
17         "access_granted": access_granted,
18         "strategy": self.strategy,
19     }

```

6.5. Identyfikacja użytkownika

Program `main.py` udostępnia interfejs CLI z parametrami `-face`, `-voice`, `-threshold` oraz `-strategy`. Fragment parsera argumentów przedstawiono w listingu 6.2.

Listing 6.2. Parser argumentów w `main.py`.

```

1 def parse_args():
2     parser = argparse.ArgumentParser(
3         description="MultiVerify      identyfikacja multimodalna"
4     )
5     parser.add_argument("--face", required=True,
6         help="Sciezka do zdjecia probe")
7     parser.add_argument("--voice", required=True,
8         help="Sciezka do nagrania probe")
9     parser.add_argument("--threshold", type=float, default=0.7)
10    parser.add_argument("--strategy",
11        choices=FusionEngine.STRATEGIES, default="weighted")
12    return parser.parse_args()

```

6.6. Rejestracja

Skrypt `register.py` automatycznie przetwarza wszystkie podkatalogi w `data/users/`, generując pliki `.npy` z embeddingami. Dzięki temu identyfikacja nie wymaga ponownego przetwarzania surowych plików multimedialnych.

6.7. Testy

Zaimplementowano testy jednostkowe modułu fuzji oraz funkcji podobieństwa. Uruchomienie: `pytest`. W momencie opracowania dokumentacji wszystkie 11 testów przechodziło pomyślnie.

7. Wyniki i eksperymenty

7.1. Konfiguracja eksperymentu

Testy przeprowadzono na bazie czterech użytkowników zarejestrowanych w systemie. Dla każdej osoby wykorzystano trzy zdjęcia twarzy i trzy nagrania głosu. Jako dane probe przyjmowano pliki trzeciej próbki (`face_3`, `voice_3`).

Porównano warianty:

- `face_only` — $w_f = 1,0$, $w_v = 0,0$,
- `voice_only` — $w_f = 0,0$, $w_v = 1,0$,
- `fusion_50_50`, `fusion_60_40`, `fusion_70_30`.

Skrypt `scripts/run_experiments.py` generuje plik `docs/experiment_results.csv`. Metryki FAR, FRR i accuracy obliczane są skryptem `compute_metrics.py` (do uzupełnienia po finalnych eksperymentach).

7.2. Przykład identyfikacji

Tabela 7.1 przedstawia wyniki identyfikacji dla probe użytkownika `kacper` (strategia `weighted`, $\tau = 0,7$).

Tabela 7.1. Wyniki identyfikacji — probe użytkownika `kacper`.

Użytkownik	Twarz	Głos	Fuzja
kacper	1,00	1,00	1,0000
kamil	0,53	0,98	0,7115
krystian	0,15	0,98	0,4829
marcel	0,17	0,97	0,4880

System poprawnie wskazał użytkownika `kacper` jako najlepsze dopasowanie. Jednocześnie zauważono wysokie podobieństwo głosu dla użytkowników obcych (np. `kamil`: 0,98), co wynika z ograniczeń MFCC.

7.3. Analiza wyników

7.3.1. Modalność twarzy

Moduł oparty na Facenet zapewnił wyraźną separację tożsamości — wyniki twarzy dla użytkowników obcych były niskie (0,15–0,53). Modalność ta okazała się najbardziej rozróżniająca w badanej bazie.

7.3.2. Modalność głosu

Wyniki głosu dla różnych użytkowników były wysokie (często $> 0,9$), nawet gdy mówca był inny. Potwierdza to, że proste MFCC bez modelu rozpoznawania mówcy słabo rozróżnia tożsamości w małej grupie mówców męskich o podobnych nagraniach.

7.3.3. Fuzja multimodalna

Fuzja 60/40 pozwoliła zachować wysoki wynik dla prawdziwego użytkownika (1,0) i jednocześnie wykorzystać niski wynik twarzy przy odrzucaniu obcych w niektórych przypadkach. Strategia and dodatkowo ogranicza fałszywe akceptacje wymagając progu w obu modalnościach.

7.3.4. Wpływ progu

Podniesienie progu τ (np. do 0,85) zmniejsza FAR kosztem wzrostu FRR. Wykres zależności FAR/FRR od progu należy wygenerować na podstawie pełnej macierzy eksperymentów (plik docs/charts/threshold_sweep.png).

7.4. Analiza bezpieczeństwa

7.4.1. Zagrożenia

- **Atak prezentacji (spoofing)** — użycie zdjęcia zamiast żywej twarzy; system nie zawiera detekcji żywotności (liveness).
- **Replay głosu** — odtworzenie nagrania; MFCC nie rozróżnia autentycznej mowy od nagrania.
- **Atak jedną modalnością** — przy dominacji słabej modalności głosu impostor może uzyskać wynik powyżej progu (np. kamil: fuzja 0,71 przy probe kacpera).
- **Jakość danych** — uszkodzone pliki MP3 mogą uniemożliwić rejestrację.

7.4.2. Środki zaradcze

Zastosowano m.in.: fuzję dwóch modalności, regułę AND, konfigurowalny próg, walidację istnienia plików wejściowych. W środowisku produkcyjnym zalecałoby się modele anty-spoofing, silniejszy model głosu oraz weryfikację 1:1 zamiast samego rankingu 1:N.

7.5. Podsumowanie eksperymentów

Eksperymenty potwierdzają poprawne działanie pipeline'u multimodalnego. Największe ograniczenie stanowi modalność głosu. Fuzja z twarzą kompensuje ten problem w testach identyfikacji właściwego użytkownika, lecz wymaga strojenia progu w celu ograniczenia FAR.

Uwaga: ostateczne wartości FAR/FRR dla wszystkich wariantów należy uzupełnić po uruchomieniu `run_experiments.py` i `compute_metrics.py`.

8. Wnioski końcowe

8.1. Podsumowanie

W ramach projektu opracowano działający system multimodalnej identyfikacji użytkownika **Multi-Verify**, łączący rozpoznawanie twarzy (DeepFace/Facenet) z analizą głosu (MFCC). Zaimplementowano pełny cykl: rejestrację, identyfikację 1:N, fuzję wyników oraz framework eksperymentalny.

8.2. Najważniejsze obserwacje

- Modalność twarzy zapewnia wysoką rozdzielczość tożsamości w testach zespołu.
- Modalność głosu oparta na MFCC wykazuje wysokie podobieństwo między różnymi użytkownikami — wymaga to uwagi przy doborze progu.
- Fuzja multimodalna poprawia odporność decyzji względem pojedynczej słabszej cechy.
- Strategia `and` jest bardziej konserwatywna i zmniejsza ryzyko fałszywej akceptacji.

8.3. Ocena skuteczności

System spełnia założenia projektu demonstracyjnego: poprawnie identyfikuje zarejestrowanych użytkowników na danych testowych zespołu. Nie jest gotowy do wdrożenia produkcyjnego bez dodatkowych mechanizmów bezpieczeństwa i silniejszego modelu głosu.

8.4. Możliwości rozwoju

Kierunki dalszego rozwoju:

- integracja z kamerą i mikrofonem w czasie rzeczywistym,
- zastąpienie MFCC modelem x-vector lub podobnym,
- detekcja żywotności (anti-spoofing),
- automatyczne strojenie progu i wag na zbiorze walidacyjnym,
- tryb weryfikacji 1:1 (logowanie do konkretnego konta).

8.5. Podział pracy

- **Kacper Ręczak** — architektura systemu, moduł twarzy, fuzja, `main.py`, `register.py`, rozdziały 1, 2, 4, 5 (twarz i fuzja), 6.
- **Kamil Szopniewski** — moduł głosu, eksperymenty, metryki FAR/FRR, wykresy, rozdziały 3, 5 (głos), 7, 8, bibliografia.

Bibliografia

- [1] Apple Inc. Face id and touch id security. <https://support.apple.com/guide/security/face-id-touch-id-sec063891067/web>, 2024. Dostęp: 11.06.2026.
- [2] Steven Davis and Paul Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.
- [3] Anil K. Jain, Patrick Flynn, and Arun A. Ross. *Handbook of Biometrics*. Springer, 2008.
- [4] Anil K. Jain, Arun Ross, and Salil Prabhakar. An introduction to biometric recognition. *IEEE Transactions on Circuits and Systems for Video Technology*, 14(1):4–20, 2004.
- [5] Brian McFee et al. *librosa: Audio and Music Signal Analysis in Python*, 2024. Dostęp: 11.06.2026.
- [6] Arun Ross and Anil K. Jain. Information fusion in biometrics. *Pattern Recognition Letters*, 24(13):2115–2125, 2006.
- [7] Florian Schroff, Dmitry Kalenichenko, and James Philbin. FaceNet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 815–823, 2015.
- [8] Sefik Ilkin Serengil and Alper Ozpinar. Deepface: A lightweight face recognition and facial attribute analysis framework. <https://github.com/serengil/deepface>, 2020. Dostęp: 11.06.2026.

Spis rysunków

4.1	Schemat blokowy systemu MultiVerify.	12
-----	--	----

Spis listingów

6.1	Fragment silnika fuzji (<code>FusionEngine</code>).....	16
6.2	Parser argumentów w <code>main.py</code>	17